

Codex Enterprise Rollout Design

Linton Labs | governed expansion from a billing-platform pilot

Recommendation

Adopt a staged rollout beginning with the legacy billing repository and two named engineering teams. Start with low-friction local surfaces, enforce review and workspace isolation, and expand only when evidence shows safe adoption, reliable verification, and support readiness. Codex accelerates implementation; humans retain ownership of approvals, merges, releases, and production risk.

Explicit decisions

Decision	Default for phase 1
Surface strategy	IDE extension for day-to-day edits; Codex app for review/orchestration; CLI for reproducible scripted work. Worktrees are mandatory for parallel or risky changes; cloud tasks are forbidden for proprietary billing code until policy approval.
Model strategy	Use the configured general model for most work; use an API-key workflow only with approved credentials and data controls. Escalate reasoning for architecture/debugging; use Spark only for narrow, low-latency loops with tighter verification.
Context continuity	Checkpoint at task boundaries, before compaction, and after failed attempts. Preserve decisions, changed files, tests, and proof gaps in the task record; recover by restarting from the last verified checkpoint.
Governance bundle	Standardize <code>config.toml</code> , <code>AGENTS.md</code> , and allowlisted <code>.rules</code> ; require verification commands, review guidance, a definition of done, and a secure-PR skill for billing work.
Cloud policy	Cache approved dependencies through setup scripts; keep secrets out of prompts and repositories; use environment variables or the approved secret store; internet access is off by default and allowlisted per task.
Integrations and admin	Begin with manual <code>@codex review</code> ; delegate Slack/Linear drafting only, never approvals or production actions. Enable local/cloud access through RBAC groups and track analytics separately from compliance logs.

Guardrails and recovery

Guardrails: sandboxed execution with least privilege; no secrets in prompts or generated code; internet off by default; mandatory human review, tests, and customer-owned merge/release decisions; log task identity, approvals, tool use, and artifacts. Escalate production access, destructive actions, unsupported integrations, or a request to bypass review.

Recovery patterns: (1) If a long-running task drifts or fails, stop, capture the last verified checkpoint, revert unverified changes, and resume with a smaller scoped task. (2) If a surface, model, or cloud environment is unavailable, switch to the approved local fallback, preserve the decision record, rerun validation, and notify the named support owner.

Rollout gates and monitoring

Gate 1: one billing repo, two teams, named reviewers, and baseline metrics. Gate 2: expand to adjacent repos only after verification pass rate, review burden, rollback readiness, and policy adherence are acceptable. Gate 3: consider CI/CD or cloud expansion after repeated evidence, RBAC coverage, support runbook validation, and audit-log completeness. Monitor adoption, verification outcomes, exception/approval rates, secret and internet-policy violations, support tickets, and time to recover.